

Stage 1: Find and critique a dataset

Dataset Selection

The dataset selected for this project is the HDB Resale Flat Prices dataset (1990-1999) from data.gov.sg. This dataset comprises transaction-level records of HDB (Housing & Development Board) resale flat transactions in Singapore over 10 years. Each row represents a resale transaction with attributes including location (town), flat specifications (type, model, floor area), and temporal information (year, month), as well as the resale prices.

The dataset represents transaction-level data rather than individual buyers or sellers. This makes it suitable for analysing housing market trends, price variations across locations, and structural factors influencing resale prices over time.

The primary motivation for selecting this dataset is its ability to support analytical queries such as:

- How resale prices change over time,
- How prices differ across towns and flat types,
- Which combinations of attributes are associated with higher resale prices.

Motivation

This dataset is of particular interest because the 1990s represented a period of significant economic expansion and urban development in Singapore. Analyzing this data allows for an investigation into how housing in the market responded to these shifts. This database approach worked well due to the volume of data, spanning years of transactions; a simple spreadsheet sorting is inefficient for complex multidimensional queries involving joins between locations, flat models, and temporal trends.

Data Quality

The dataset is published by the Singapore government via data.gov.sg, ensuring a high level of data credibility, accuracy, and reliability. The data is structured consistently, with clearly defined attributes and no obvious duplication of records. Some attributes, such as `floor_area_sqm`, may contain missing or zero values. These were handled during data ingestion by filtering or allowing NULLs where appropriate.

Terms of Use and Data Source

The dataset is openly available on data.gov.sg and is released under Singapore's Open Data License. This allows free use, reuse, and redistribution with no restrictions. As such, the dataset satisfies the requirement for open, accessible data with no limitation on usage.

Level of Detail

The dataset provides a high level of detail, with each row representing a unique resale transaction. The high level of granularity allows meaningful aggregation and analysis across multiple dimensions, including:

Attribute	Description	Data type	Example
month	Transaction approval date	String (YYYY-MM)	1990-01
town	Location of the flat	String	"ANG MO KIO"
flat_type	Category of flat	String	"4 ROOM"
flat_model	Specific model designation	String	"Model A"
floor_area_sqm	Size in square meters	Decimal	90.00
resale_price	Transaction price	Integer(SGD)	150000
lease_commencement_date	Year lease started	Integer	1985

Table 1: Original Dataset

Documentation

Basic documentation is provided through the website describing attributes. However, some attributes, such as flat models, require domain understanding to interpret correctly. This was addressed by reviewing HDB housing terminology and ensuring consistent categorisation during database normalisation.

Interrelation

The dataset is self-contained but naturally lends itself to a relational design. Attributes such as town names, flat types, and time period repeat across transactions, making them suitable candidates for separate dimension tables. This allows the data to be efficiently normalised and queried using joins.

Analytical Questions

Question 1: How have the average HDB resale prices evolved year-by-year from 1990 to 1999?

Question 2: Which towns have the highest and lowest average resale prices?

Question 3: What are the average prices for different flat types?

Question 4: What are the monthly and yearly transaction volume patterns?

Question 5: What combinations of town, flat type, and year yield the most expensive transaction?

Stage 2: Model your Data

Based on stage 1, the data is categorized into five distinct entities. The goal is to separate static dimension data, such as town names, from the dynamic fact data, such as the transactions.

Town: Represents the geographical location

- Fields: town_id (PK), town_name

Flat types: Represent the room-count classification

- Fields: flat_type_id (PK), flat_type

Flat Model: Represents the architecture model of the flat

- Fields: flat_model_id (PK), flat_model

Time: Represents the temporal dimension of the transaction

- Field: time_id (PK), year, month

Resale Transaction: The central entity recording the actual sale

- Fields: transaction_id (PK), floor_area_sqm, lease_commencement_year, resale_price.

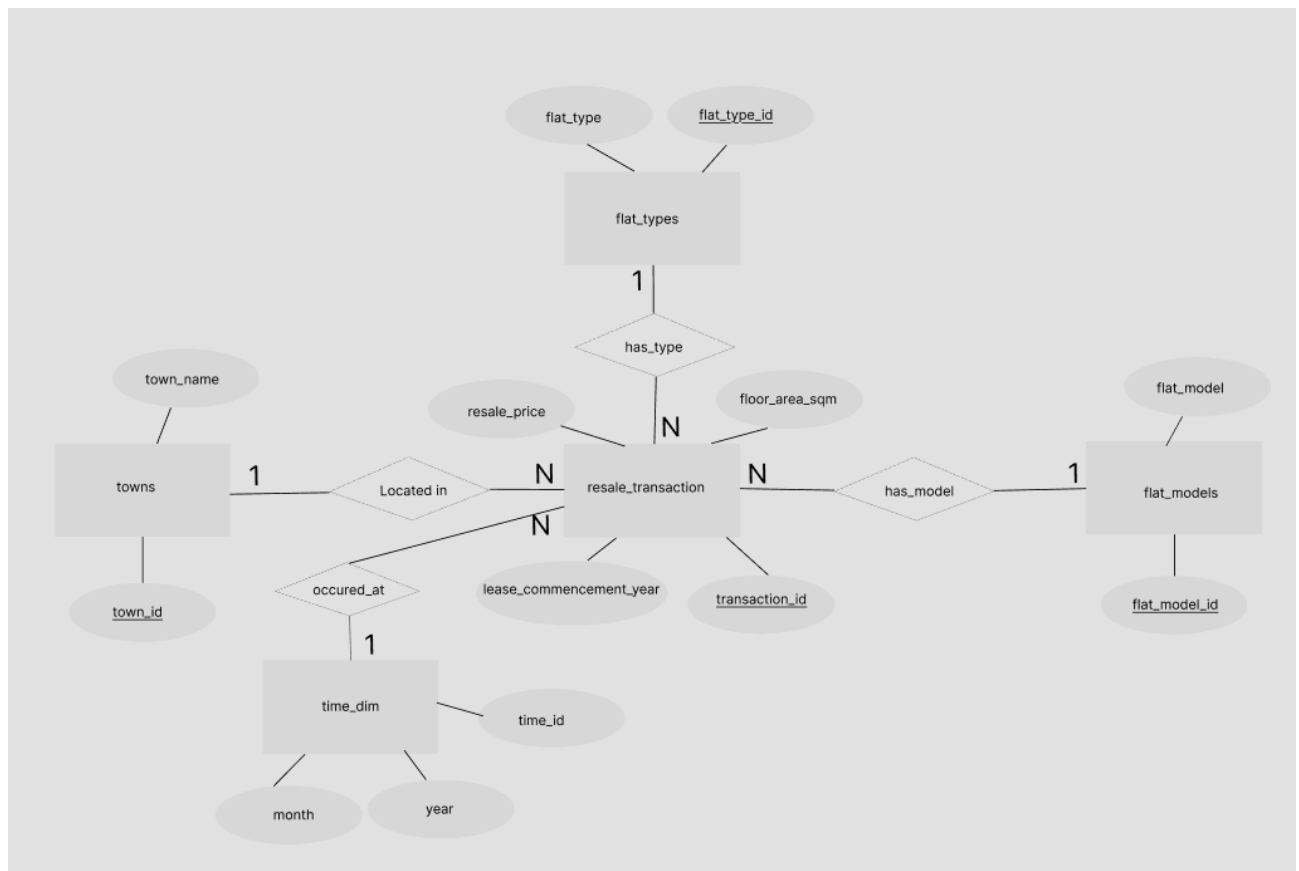


Figure1: E/R Model

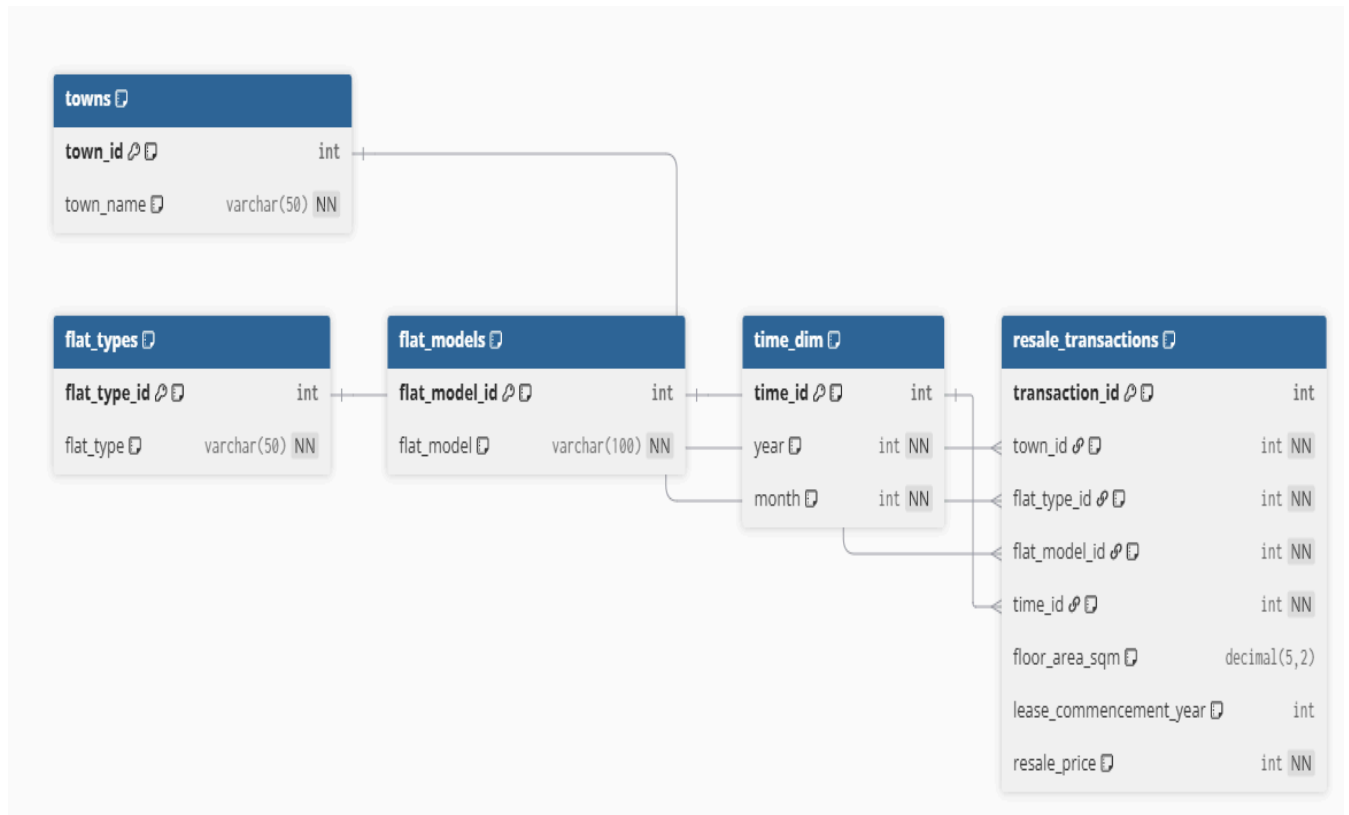


Figure 2: Database Diagram

Normalisation

The final database schema is in Boyce-Codd Normal Form (BCNF). Each table has a clearly defined primary key that uniquely identifies its records, and all non-key attributes are fully functionally dependent on that key. Repeating attributes such as towns, flat types, flat models, and time periods are stored in separate dimension tables, while the **resale_transactions** table stores only measurable values and foreign keys. This structure reduces data redundancy, ensures referential integrity, and prevents anomalies related to updates, insertions, and deletions.

Stage 3: Create the Database

This stage focuses on implementing the relational database, which was designed in Stage 2. This includes creating the database schema in MySQL, defining tables with primary and foreign key constraints, processing raw CSV data, and populating the normalised tables to support analytical queries and the web application.

The database was implemented using MySQL in the Coursera lab environment. During development, the database was dropped and recreated to ensure a clean environment, avoiding data duplication and schema conflicts that I encountered while iterating on the design.

```
DROP DATABASE IF EXISTS hdb_resale;  
CREATE DATABASE hdb_resale;  
USE hdb_resale;
```

Figure 3

This method ensured consistent and reproducible database creation during testing and development. The database schema consists of four dimension tables and one fact table, along with a staging table used for data ingestion. Each table stores a unique set of descriptive attributes.

The time_dim table enforces a constraint on (year, month) to prevent duplicate time records. The resale_transactions table stores measurable data and foreign keys linking to the dimensions table.

```
CREATE TABLE towns (  
    town_id INT AUTO_INCREMENT PRIMARY KEY,  
    town_name VARCHAR(50) NOT NULL UNIQUE  
);  
  
CREATE TABLE flat_types (  
    flat_type_id INT AUTO_INCREMENT PRIMARY KEY,  
    flat_type VARCHAR(50) NOT NULL UNIQUE  
);  
  
CREATE TABLE flat_models (  
    flat_model_id INT AUTO_INCREMENT PRIMARY KEY,  
    flat_model VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE time_dim (  
    time_id INT AUTO_INCREMENT PRIMARY KEY,  
    year INT NOT NULL,  
    month INT NOT NULL,  
    UNIQUE (year, month)  
);  
  
CREATE TABLE resale_transactions (  
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,  
    town_id INT NOT NULL,  
    flat_type_id INT NOT NULL,  
    flat_model_id INT NOT NULL,  
    time_id INT NOT NULL,  
    floor_area_sqm DECIMAL(5,2),  
    lease_commencement_year INT,  
    resale_price INT NOT NULL,  
    FOREIGN KEY (town_id) REFERENCES towns(town_id),  
    FOREIGN KEY (flat_type_id) REFERENCES flat_types(flat_type_id),  
    FOREIGN KEY (flat_model_id) REFERENCES flat_models(flat_model_id),  
    FOREIGN KEY (time_id) REFERENCES time_dim(time_id)  
);
```

Figure 4

A staging table was created to facilitate data cleaning and normalization during the ingestion of the ResaleFlatPricesBasedonApprovalDate19901999.csv file.

- Staging: Data was first loaded into the staging_resale table using LOAD DATA LOCAL INFILE to preserve the raw format
- Normalization: Dimension tables were populated using INSERT IGNORE INTO..... SELECT DISTINCT from the staging table
- Data Transformation: During the final population of resale_transaction, string manipulation(eg. SUBSTRING and TRIM) was used to convert the raw month string (eg, 1990-01) into corresponding year and month integers for the time_dim table.

```
CREATE TABLE staging_resale (  
  month VARCHAR(10),  
  town VARCHAR(50),  
  flat_type VARCHAR(50),  
  flat_model VARCHAR(100),  
  floor_area_sqm DECIMAL(5,2),  
  lease_commencement_year INT,  
  resale_price INT  
);
```

Figure 5

The dataset was loaded into the staging table using the LOAD DATA LOCAL INFILE command:

```
LOAD DATA LOCAL INFILE '/home/coder/project/csv/ResaleFlatPricesBasedonApprovalDate19901999.csv'  
INTO TABLE staging_resale  
FIELDS TERMINATED BY ','  
ENCLOSED BY ''  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

Figure 6

```
INSERT IGNORE INTO towns (town_name)  
SELECT DISTINCT town FROM staging_resale  
WHERE town IS NOT NULL;  
  
INSERT IGNORE INTO flat_types (flat_type)  
SELECT DISTINCT flat_type FROM staging_resale  
WHERE flat_type IS NOT NULL;  
  
INSERT IGNORE INTO flat_models (flat_model)  
SELECT DISTINCT flat_model FROM staging_resale  
WHERE flat_model IS NOT NULL;  
  
INSERT IGNORE INTO time_dim (year, month)  
SELECT DISTINCT  
  CAST(SUBSTRING(month, 1, 4) AS UNSIGNED) as year,  
  CAST(SUBSTRING(month, 6, 2) AS UNSIGNED) as month  
FROM staging_resale  
WHERE month IS NOT NULL AND month != '';
```

Figure 7

Critical Reflection

Positive points:

Normalization: The use of foreign keys prevented orphaned records and ensured that every transaction is linked to valid town and flat-type data.

Storage Efficiency: By using integer foreign keys rather than repeating long strings such as "JURONG WEST", the database size is significantly reduced.

Challenges faced:

Data consistency: The raw CSV contained variations in formatting for month strings.

Using a staging table was important for cleaning this data before the final production tables.

Query complexity: The database being in BCNF provided high data integrity, but increased the complexity of queries, requiring multiple JOIN operations to retrieve user-friendly reports.

Limitations:

The dataset covers only the period 1990-1999, limiting applicability to current housing trends.

Buyer-level attributes such as income or household size are not available, preventing demographic analysis.

SQL commands that answer questions identified in Stage 1/Stage 3

Question 1: "How have the average HDB resale prices evolved year-by-year from 1990 to 1999?"

To analyse long-term price trends, an aggregate query was executed to calculate the average resale price per year, along with the total number of transactions. It provides a macro-level view of Singapore's property markets' growth during the 1990s.

```
Database changed
mysql> SELECT
->     td.year,
->     ROUND(AVG(rt.resale_price), 2) as avg_price,
->     COUNT(*) as num_transactions
-> FROM resale_transactions rt
-> JOIN time_dim td ON rt.time_id = td.time_id
-> GROUP BY td.year
-> ORDER BY td.year;

+-----+-----+-----+
| year | avg_price | num_transactions |
+-----+-----+-----+
| 1990 | 83.76 | 100040 |
| 1991 | 85.77 | 102840 |
| 1992 | 87.65 | 116024 |
| 1993 | 91.06 | 144928 |
| 1994 | 92.19 | 210984 |
| 1995 | 92.39 | 218312 |
| 1996 | 91.13 | 279352 |
| 1997 | 95.21 | 254072 |
| 1998 | 95.86 | 408760 |
| 1999 | 98.34 | 462256 |
+-----+-----+-----+
10 rows in set (17.90 sec)
```

Figure 8

Observation:

- Average resale prices increased steadily over the decade
- There seems to be a likely significant upward trend peaking around 1996-1997, followed by a correction or stabilization.
- In 1990, the average resale price was approximately \$83.76k, rising consistently to about \$98.3k by 1999.
- Transaction volume also increased significantly, indicating growing market activity alongside rising prices.

Interpretation:

This upward trend suggests a strengthening resale market throughout the 1990s, likely influenced by population growth, housing demand, and economic expansion. The increasing transaction count reinforces the reliability of the observed price trend.

Question 2: "Which towns have the highest and lowest average resale prices?"

To identify spatial price variation, average resale prices were computed for each town and ranked.

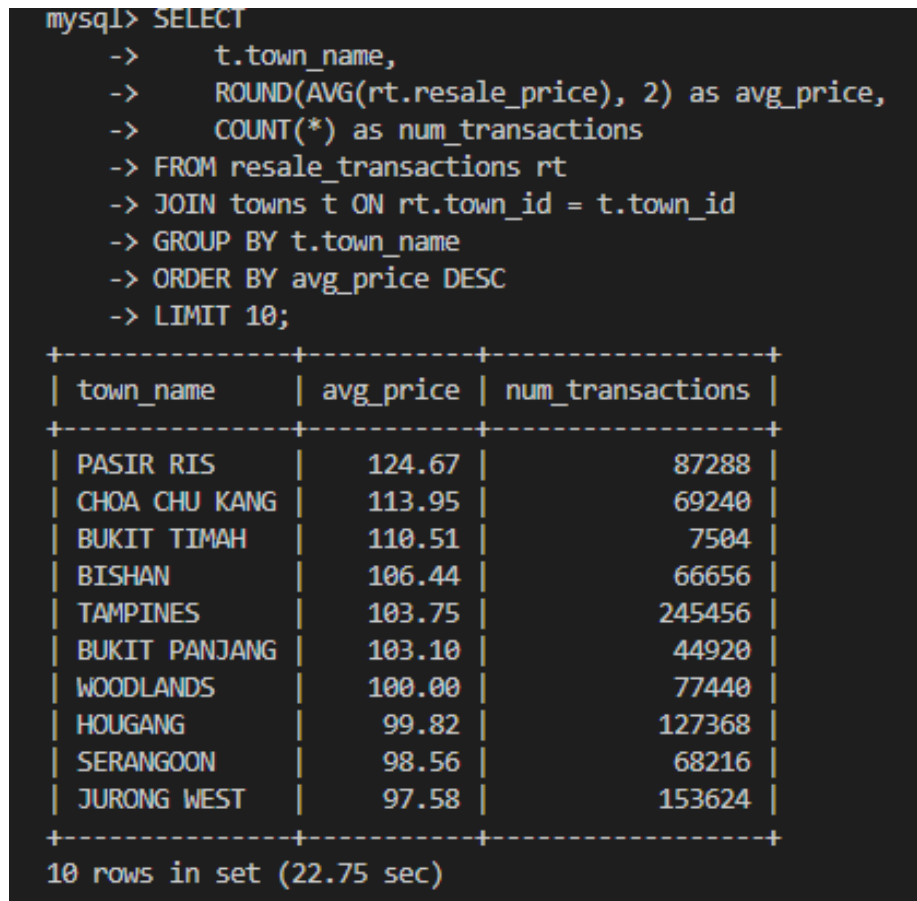


Figure 9

Findings (Top 10 by average price):

- Pasir Ris recorded the highest resale price
- Other high-priced towns include Choa Chu Kang, Bukit Timah, and Bishan
- Towns such as Jurong West and Sengkang appeared lower in the top-10 ranking, indicating relatively more affordable pricing.

Interpretation:

Price differences across towns highlight the importance of location as a key determinant of resale value. Factors such as amenities, transport links, and town maturity likely contribute to these variations.

Question 3: "What are the average prices for different flat types?"

This query analysed resale prices grouped by flat type, calculating the average, minimum, and maximum prices. There is a clear linear correlation between the number of rooms and the price, but the "Max Price" for smaller flats in prime areas sometimes overlaps with larger flats in far-flung areas.

```
mysql> SELECT
->   ft.flat_type,
->   ROUND(AVG(rt.resale_price), 2) as avg_price,
->   ROUND(MIN(rt.resale_price), 2) as min_price,
->   ROUND(MAX(rt.resale_price), 2) as max_price
-> FROM resale_transactions rt
-> JOIN flat_types ft ON rt.flat_type_id = ft.flat_type_id
-> GROUP BY ft.flat_type
-> ORDER BY avg_price DESC;
```

flat_type	avg_price	min_price	max_price
MULTI GENERATION	161.95	132	179
EXECUTIVE	148.77	138	243
5 ROOM	124.17	104	167
4 ROOM	96.55	74	135
3 ROOM	68.65	46	307
2 ROOM	45.05	37	67
1 ROOM	30.09	28	31

7 rows in set (59.36 sec)

Figure 10

Results:

- Multi-generational flats had the highest average resale price.
- Executive flats followed closely behind
- Prices decreased progressively from 5-room to 1-room flats, with 1-room flats being the most affordable.
- Location seems to be very important as well

Interpretations:

Larger flat types command higher resale prices due to greater floor area and suitability for larger households. This confirms a strong relationship between flat size/type and market value.

Another observation is that while size (flat type) is a primary price driver, location (town) acts as a powerful secondary variable.

Question 4: “What are the monthly and yearly transaction volume patterns?”

This query analysed combinations of town, flat types, and year, filtering for combinations with sufficient transaction volume. Transaction volumes typically increased toward the end of the decade, with certain months showing consistent “spikes” in activity.

```
mysql> SELECT
->     td.year,
->     td.month,
->     COUNT(*) as num_transactions
-> FROM resale_transactions rt
-> JOIN time_dim td ON rt.time_id = td.time_id
-> GROUP BY td.year, td.month
-> ORDER BY td.year, td.month;
```

year	month	num_transactions
1990	1	12056
1990	10	6480
1990	11	8720
1990	12	8160
1990	2	7448
1990	3	9200
1990	4	8376
1990	5	6224
1990	6	6592
1990	7	11240
1990	8	8624
1990	9	6920
1991	1	10024
1991	10	9464
1991	11	11056
1991	12	8400
1991	2	6528
1991	3	6832
1991	4	7360
1991	5	7560
1991	6	6688
1991	7	9976
1991	8	9304
1991	9	9648
1992	1	9616
1992	10	10824
1992	11	11008
1992	12	10360
1992	2	7632
1992	3	8904
1992	4	8536
1992	5	9120
1992	6	8472
1992	7	10352
1992	8	10032

Figure 11

Observations:

- Transaction volume increased over time, particularly in the late 1990s.
- Certain months consistently recorded higher transaction counts, suggesting mild seasonal patterns.

Interpretation:

The increasing volume reflects growing participation in the resale market. Monthly variations may be influenced by policy changes, economic cycles, or buyer behavior, although further data would be required for causal analysis. Seasonal spikes may suggest “buying season” or periods following any government policy announcements.

Question 5: “What combinations of town, flat type, and year yield the most expensive transaction?”

This query analysed combinations of town, flat-type, and year, filtering for combinations with sufficient transaction volume. This multi-dimensional query joins four tables to find the “pinnacle” of the market, filtering for combinations with at least 10 transactions to ensure statistical relevance.

```
mysql> SELECT
->     t.town_name,
->     ft.flat_type,
->     td.year,
->     AVG(rt.resale_price) as avg_price,
->     COUNT(*) as num_transactions
-> FROM resale_transactions rt
-> JOIN towns t ON rt.town_id = t.town_id
-> JOIN flat_types ft ON rt.flat_type_id = ft.flat_type_id
-> JOIN time_dim td ON rt.time_id = td.time_id
-> GROUP BY t.town_name, ft.flat_type, td.year
-> HAVING COUNT(*) >= 10
-> ORDER BY avg_price DESC
-> LIMIT 20;
```

town_name	flat_type	year	avg_price	num_transactions
WOODLANDS	EXECUTIVE	1999	171.5208	5760
WOODLANDS	EXECUTIVE	1998	170.8486	1480
YISHUN	MULTI GENERATION	1992	168.5625	128
YISHUN	MULTI GENERATION	1995	167.4211	152
YISHUN	MULTI GENERATION	1994	166.1667	144
TAMPINES	MULTI GENERATION	1995	166.0000	16
YISHUN	MULTI GENERATION	1993	165.6154	208
BISHAN	MULTI GENERATION	1999	165.3333	48
YISHUN	MULTI GENERATION	1998	165.0625	128
BISHAN	MULTI GENERATION	1992	165.0000	24
YISHUN	MULTI GENERATION	1996	164.8750	192
YISHUN	MULTI GENERATION	1999	163.9231	104
BISHAN	MULTI GENERATION	1994	163.7500	64
BISHAN	MULTI GENERATION	1995	163.0000	48
YISHUN	MULTI GENERATION	1997	162.7895	152
BISHAN	MULTI GENERATION	1998	161.1429	56
BISHAN	MULTI GENERATION	1993	160.7500	96
TAMPINES	MULTI GENERATION	1992	160.6667	24
BISHAN	MULTI GENERATION	1996	159.8333	48
BISHAN	MULTI GENERATION	1997	159.2500	64

20 rows in set (1 min 31.15 sec)

Figure 12

Observation:

- Executive and Multi-generation flats in towns such as Woodlands, Yishun, and Bishan during the late 1990s recorded the highest average prices.
- The most expensive combination occurred predominantly in 1997-1999, aligning with the overall upward price trend.

Interpretation:

High resale prices are driven by a combination of locations, flat type, and time period, rather than a single factor. This demonstrates the value of a multidimensional database design for uncovering complex relationships.

Stage 4: Web Application

A simple web application was developed using [Node.js](#) and Express to demonstrate integration with a normalised HDB resale database. The application connects to a MySQL backend and executes analytical SQL queries through HTTP routes.

The web interface offers multiple analytical views, including average resale prices by year and comparisons of the most expensive towns, as well as the average price by flat type. Query results are dynamically retrieved from the database and rendered as HTML tables in the browser.

```
const express = require('express');
const path = require('path');
const db = require('./db');

const app = express();
const PORT = 3000;

app.use(express.static(path.join(__dirname, 'webapppage')));

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'webapppage', 'index.html'));
});
```

Figure 13

Express allows the application to define URL routes that correspond to different analytical queries, such as /avg-price-year and /top-towns. Express is used to create the web server and define routes. Path ensures file paths are resolved correctly across environments.

Db handles database connectivity. The mysql2 linear is used to connect the [Node.js](#) application to MySQL. This ensures a successful connection to the hdb_resale database before handling user requests.

```
JS db.js > [e] db
1  const mysql = require('mysql2');
2
3  const db = mysql.createConnection({
4    host: 'localhost',
5    user: 'root',
6    password: '',
7    database: 'hdb_resale'
8  });
9
10 db.connect(err => {
11   if (err) {
12     console.error('Database connection failed:', err)
13     return;
14   }
15   console.log('Connected to MySQL database');
16 });
17
18 module.exports = db;
```

Figure 14

Due to the application running within the Coursera Lab environment, routes are accessed via /proxy/3000/. This home page provides a clear title, a brief description of the application, and links to the database data.

This screenshot displays the home page of the web application. The page serves as the entry point for users and provides navigation links to the available analytical views. This demonstrates that the Express server is correctly serving HTML content and that users can interact with the web interface to access database-driven analyses.

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta http-equiv="X-UA-Compatible" content="IE=edge">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <base href="/proxy/3000/">
8      <link rel="stylesheet" href="style.css">
9      <title>HDB Resale Analysis</title>
10 </head>
11 <body>
12     <h1>HDB Resale Analysis</h1>
13     <p>Simple web application on analysing HDB resale data</p>
14
15     <ul>
16         <li><a href="/proxy/3000/avg-price-year">Average Resale Price By Year</a></li>
17         <li><a href="/proxy/3000/top-towns">Top 10 Most Expensive Towns</a></li>
18         <li><a href="/proxy/3000/avg-price-flat-type">Average Price by Flat Type</a></li>
19     </ul>
20 </body>
21 </html>
```

Figure 15

Each route in the web application directly maps to an analytical SQL query defined in stage 3, demonstrating how database queries can be operationalized within a simple application interface.

This route executes a SQL query that groups transactions by year, calculates the average resale price, and counts the number of transactions per year.

This screenshot displays the average HDB resale price by year. The data is retrieved dynamically from the database using an SQL aggregation query that joins the resale transactions table with the time dimension. The results are grouped by year and rendered as an HTML table, demonstrating the successful execution of SQL queries through the web application.

```

app.get('/avg-price-year', (req, res) => {
  const sql = `
    SELECT
      td.year,
      ROUND(AVG(rt.resale_price), 2) as avg_price,
      COUNT(*) as num_transactions
    FROM resale_transactions rt
    JOIN time_dim td ON rt.time_id = td.time_id
    GROUP BY td.year
    ORDER BY td.year;
  `;

  db.query(sql, (err, results) => {
    if (err) {
      res.status(500).send('Database error');
      return;
    }

    let html = `
      <h1>Average Resale Price by Year</h1>
      <table border="1">
        <tr>
          <th>Year</th>
          <th>Average Price</th>
          <th>Transaction</th>
        </tr>
      `;

    results.forEach(r => {
      html += `
        <tr>
          <td>${r.year}</td>
          <td>${r.avg_price}</td>
          <td>${r.num_transactions}</td>
        </tr>
      `;
    });

    html += `
      </table>
      <br><a href="/proxy/3000/">Back to home</a>
    `;
  });
});

```

Figure 16

Average Resale Price by Year

Year	Average Price	Transaction
1990	83.76	100040
1991	85.77	102840
1992	87.65	116024
1993	91.06	144928
1994	92.19	210984
1995	92.39	218312
1996	91.13	279352
1997	95.21	254072
1998	95.86	408760
1999	98.34	462256

[Back to home](/proxy/3000/)

Figure 17

This route presents the top 10 most expensive towns based on average resale price. The view is generated by executing an SQL query that joins the resale transaction table with the towns dimensions and applies grouping, ordering, and limiting operations. This confirms that the web application can perform multi-table joins and displays comparative analytical results.

Top 10 Most Expensive Towns

Year	Average Price	Transaction
PASIR RIS	124.67	87288
CHOA CHU KANG	113.95	69240
BUKIT TIMAH	110.51	7504
BISHAN	106.44	66656
TAMPINES	103.75	245456
BUKIT PANJANG	103.10	44920
WOODLANDS	100.00	77440
HOUGANG	99.82	127368
SERANGOON	98.56	68216
JURONG WEST	97.58	153624

[Back to home](#)

Figure 18

```
app.get('/top-towns', (req, res) => {
  const sql = `
    SELECT
      t.town_name,
      ROUND(AVG(rt.resale_price), 2) as avg_price,
      COUNT(*) as num_transactions
    FROM resale_transactions rt
    JOIN towns t ON rt.town_id = t.town_id
    GROUP BY t.town_name
    ORDER BY avg_price DESC
    LIMIT 10;
  `;

  db.query(sql, (err, results) => {
    if (err) {
      res.status(500).send('Database error');
      return;
    }

    let html = `
    <h1>Top 10 Most Expensive Towns</h1>
    <table border="1">
    <tr>
      <th>Year</th>
      <th>Average Price</th>
      <th>Transaction</th>
    </tr>
    `;

    results.forEach(r => {
      html += `
      <tr>
        <td>${r.town_name}</td>
        <td>${r.avg_price}</td>
        <td>${r.num_transactions}</td>
      </tr>
    `;
    });
  });
});
```

Figure 19

This query analyses how HDB resale prices vary across different flat types by aggregating transaction-level data. The query groups resale transactions by flat types and computes the average, minimum, and maximum resale prices for each category. This allows comparisons of price levels across housing sizes, highlighting how larger flat types, such as Executive and Multi-generation flats, command higher resale prices compared to smaller flat types like 2-room and 3-room flats. Implementing this query in the web application allows users to easily explore structural price differences associated with flat configurations.

Average Price by Flat type

Year	Average Price	Transaction
MULTI GENERATION	161.95	undefined
EXECUTIVE	148.77	undefined
5 ROOM	124.17	undefined
4 ROOM	96.55	undefined
3 ROOM	68.65	undefined
2 ROOM	45.05	undefined
1 ROOM	30.09	undefined

[Back to home](#)

Figure 20

References

Data.gov.sg. (n.d.). *Singapore Open Data Licence*. [online] Available at:

<https://data.gov.sg/open-data-licence>.

data.gov.sg. (2025). *data.gov.sg*. [online] Available at:

https://data.gov.sg/datasets?topics=housing&resultId=d_ebc5ab87086db484f88045b47411ebc5.

Moataz-Elmesmary (2020). *Data-Science-Roadmap/Books/Abraham Silberschatz, Henry Korth and S. Sudarshan - Database System Concepts. 7-McGraw-Hill Education (2020).pdf at main · Moataz-Elmesmary/Data-Science-Roadmap*. [online] GitHub.

Available at:

[https://github.com/Moataz-Elmesmary/Data-Science-Roadmap/blob/main/Books/Abraham%20Silberschatz%2C%20Henry%20Korth%20and%20S.%20Sudarshan%20-%20Database%20System%20Concepts.%207-McGraw-Hill%20Education%20\(2020\).pdf](https://github.com/Moataz-Elmesmary/Data-Science-Roadmap/blob/main/Books/Abraham%20Silberschatz%2C%20Henry%20Korth%20and%20S.%20Sudarshan%20-%20Database%20System%20Concepts.%207-McGraw-Hill%20Education%20(2020).pdf).

<https://dbdiagram.io/home>

www.w3schools.com. (n.d.). *SQL JOIN*. [online] Available at:

https://www.w3schools.com/sql/sql_ref_join.asp.

Oracle Corporation (2019). *MySQL :: MySQL 8.0 Reference Manual*. [online]

Mysql.com. Available at: <https://dev.mysql.com/doc/refman/8.0/en/>.